GitHub Action CI/CD Framework - Documentation Technique

1. Architecture Générale & Flux Principal

1.1 Flux d'Exécution Principal

```
┌──────────────────────────────────────────────────────────────────┐
│                     GITHUB ACTION TRIGGER                        │
└──────────────────────────┬───────────────────────────────────────┘
                           │
                           │
          ┌────────────────┴────────────────┐
          │                                 │
     [PRE-FLIGHT]                      [MAIN WORKFLOW]
          │                                 │
     ├─ Load Config                 ├─ Checkout Code
     ├─ Validate Token              ├─ Setup Environment
     ├─ Security Checks             ├─ PRE-HOOK (Webhook)
     ├─ Environment Lock            │
     └─ Release Blockers            ├─ PRE-PLUGIN (Script)
                                    │
                           ┌────────┴──────────┐
                           │                   │
                      [EXECUTE MODULES]        │
                           │                   │
          ┌────────────────┴────────────────┴──────────┐
          │                │            │             │
       [BUILD]         [TESTS]      [SECURITY]    [RELEASE]
          │                │            │             │
     ├─ Build App     ├─ Unit Tests ├─ Lint    ├─ Detect Changes
     ├─ Artifacts     ├─ Integration├─ Scan    ├─ Bump Version
     └─ Checksum      ├─ Coverage   ├─ SAST    ├─ Generate Changelog
                      └─ Bench      └─ Deps    ├─ Create Tag
                                               ├─ Create Release
                                               └─ Deploy
          │                │            │             │
          └────────────────┴────────────┴─────────────┘
                           │
                      [POST-HOOK] (Webhook)
                           │
                      [POST-PLUGIN] (Script)
                           │
                     [GENERATE REPORTS]
                           │
                ├─ Compliance Report
                ├─ Deployment Report
                ├─ Audit Log Export
                └─ Artifacts Upload
                           │
                      [COMPLETE / ROLLBACK]
```

2. INPUTS DE L'ACTION

2.1 Configuration Générale

Input	Type	Défaut	Description	Exemple
`config-source`	choice	`file`	Source de configuration : `file` / `input` / `default`	`file`
`config-path`	string	`.instantrelease.yml`	Chemin du fichier de configuration	`.github/instantrelease.yml`
`config-schema-check`	boolean	`false`	Valider le YAML contre schéma interne	`true`
`debug`	boolean	`false`	Activer logs détaillés	`true`
`dry-run`	boolean	`false`	Mode simulation (pas de push/tag)	`true`

2.2 Authentification & Sécurité

Input	Type	Défaut	Description	Exemple
`github-token`	string	`secrets.GITHUB_TOKEN`	Token GitHub avec `contents: write`	`${{ secrets.GITHUB_TOKEN }}`
`git-user-name`	string	`github-actions[bot]`	Nom utilisateur Git pour commits	`Release Bot`
`git-user-email`	string	`github-actions[bot]@github.com`	Email utilisateur Git	`bot@example.com`
`ssh-key`	string	-	Clé SSH pour operations Git avancées	`${{ secrets.SSH_KEY }}`
`gpg-key`	string	-	Clé GPG pour signer tags/commits	`${{ secrets.GPG_KEY }}`
`gpg-passphrase`	string	-	Passphrase pour clé GPG	`${{ secrets.GPG_PASSPHRASE }}`
`validate-token`	boolean	`true`	Valider permissions du token	`true`
`token-scope-check`	boolean	`true`	Vérifier scopes du token (repo, workflow, etc.)	`true`
`secret-scan-enabled`	boolean	`false`	Activer scan de secrets avec trufflehog	`true`

2.3 Git & Branching

Input	Type	Défaut	Description	Exemple
`trigger-branches`	string	`main,develop`	Branches d'origine (comma-separated)	`main,staging,develop`
`target-branch`	string	`main`	Branche cible pour commits auto	`main`
`auto-commit`	boolean	`true`	Effectuer commit automatique	`true`
`auto-commit-message`	string	`chore: release version {version}`	Template message commit	`release: {version}`
`auto-push`	boolean	`true`	Push automatique après commit	`true`
`branch-protection-aware`	boolean	`false`	Créer PR si push impossible	`true`
`branch-policy-check`	boolean	`true`	Vérifier stratégie merge (squash/merge/ff)	`true`
`review-enforcement`	boolean	`false`	Bloquer si PR non approuvée	`true`

2.4 Versioning

Input	Type	Défaut	Description	Exemple
`bump`	choice	`auto`	Type bump : `major` / `minor` / `patch` / `auto`	`auto`
`version`	string	-	Version spécifique à forcer (override bump)	`1.2.3`
`initial`	string	`0.1.0`	Version initiale si aucune trouvée	`1.0.0`
`detect-bump-type`	boolean	`true`	Scanner commits pour bump auto	`true`
`major-indicator`	string	`BREAKING CHANGE`	Pattern pour major bump	`[BREAKING]`
`minor-indicator`	string	`feat:`	Pattern pour minor bump	`feature:`
`patch-indicator`	string	`fix:`	Pattern pour patch bump	`bugfix:`
`version-prefix`	string	`v`	Préfixe tag version	`v`

Input	Type	Défaut	Description	Exemple
`semantic-convention`	choice	`angular`	Convention commits : `angular` / `conventional` / `custom`	`angular`
`verify-tag`	boolean	`true`	Vérifier si tag existe déjà	`true`
`idempotent-check`	boolean	`true`	Arrêter si tag existe	`true`
`sync-version`	string	-	Fichiers version à synchroniser (comma-separated)	`package.json,Chart.yaml,VERSION`

2.5 Changelog & Release Notes

Input	Type	Défaut	Description	Exemple
`generate-changelog`	boolean	`true`	Générer changelog	`true`
`changelog-file`	string	`CHANGELOG.md`	Chemin fichier changelog	`docs/CHANGELOG.md`
`changelog-template`	string	-	Template custom Markdown pour changelog	`./templates/changelog.hbs`
`append-changelog`	choice	`prepend`	Mode ajout : `prepend` / `append` / `replace`	`prepend`
`extract-changelog`	boolean	`false`	Retourner uniquement dernière entrée	`true`
`changelog-to-release-notes`	boolean	`true`	Convertir changelog en notes GitHub Release	`true`

2.6 Tagging & Releases

Input	Type	Défaut	Description	Exemple
`create-tag`	boolean	`true`	Créer tag Git	`true`
`create-release`	boolean	`true`	Créer GitHub Release	`true`
`release-draft`	boolean	`false`	Release en mode brouillon	`false`
`release-prerelease`	boolean	`false`	Marquer comme pré-release (alpha/beta)	`false`
`prerelease-suffix`	string	-	Suffixe pré-release (`alpha`,`beta`,`rc`)	`rc`
`tag-delete-on-failure`	boolean	`true`	Supprimer tag en cas d'échec	`true`
`package-release`	boolean	`false`	Créer changelog/tag/release par package	`true`
`monorepo-mode`	boolean	`false`	Mode monorepo (packages multiples)	`true`

2.7 Build & Artifacts

Input	Type	Défaut	Description	Exemple
`build-enabled`	boolean	`true`	Activer étape build	`true`
`build-command`	string	`npm run build`	Commande build personnalisée	`make build`
`build-artifact-path`	string	`dist`	Chemin artefacts après build	`build/`
`upload-artifact`	boolean	`false`	Upload changelog/version.json en artifact	`true`
`artifact-retention-days`	number	`30`	Durée conservation artefacts	`60`
`generate-version-file`	boolean	`false`	Créer fichier VERSION ou .version	`true`
`generate-checksum`	boolean	`false`	Générer fichiers .sha256	`true`
`generate-sbom`	boolean	`false`	Générer Software Bill of Materials (SPDX)	`true`
`artifact-signing`	boolean	`false`	Signer cryptographiquement artefacts	`true`

2.8 Tests & Qualité

Input	Type	Défaut	Description	Exemple
`tests-enabled`	boolean	`true`	Activer étape tests	`true`
`unit-tests-command`	string	`npm test`	Commande tests unitaires	`pytest tests/`
`integration-tests-command`	string	-	Commande tests d'intégration	`npm run test:integration`
`coverage-threshold`	number	`80`	Seuil minimum couverture (%)	`90`
`coverage-file-path`	string	`.coverage-summary.json`	Chemin rapport couverture	`coverage/coverage-summary.json`
`performance-benchmark-enabled`	boolean	`false`	Comparer perfs vs release précédente	`true`
`performance-benchmark-file`	string	`benchmark.json`	Fichier benchmark	`perf/results.json`
`lint-enabled`	boolean	`true`	Activer linting	`true`
`lint-command`	string	`npm run lint`	Commande lint	`eslint src/`
`lint-report-parse`	boolean	`false`	Parser rapport lint/sonar	`true`
`lint-report-path`	string	-	Chemin rapport lint	`reports/lint.json`
`lint-score-threshold`	number	`80`	Score lint minimum	`85`

2.9 Security & Scanning

Input	Type	Défalt	Description	Exemple
`security-checks-enabled`	boolean	`true`	Activer contrôles sécurité	`true`
`dependency-scan-enabled`	boolean	`true`	Scan dépendances (npm audit, pip-audit)	`true`
`dependency-scan-tool`	choice	`auto`	Outil : `auto` / `npm` / `pip` / `snyk` / `osv-scanner`	`snyk`
`dependency-severity-threshold`	choice	`high`	Niveau minimum alert : `low` / `medium` / `high` / `critical`	`high`
`sast-enabled`	boolean	`false`	Activer analyse statique (SAST)	`true`
`sast-tool`	choice	-	Outil SAST : `sonarqube` / `semgrep` / `codeql`	`codeql`
`commit-signature-check`	boolean	`false`	Valider signatures GPG commits	`true`
`tag-signature`	boolean	`false`	Signer tags cryptographiquement	`true`
`supply-chain-check`	boolean	`false`	Vérifier dépendances (hash, provenance)	`true`
`sbom-format`	choice	`spdx`	Format SBOM : `spdx` / `cyclonedx`	`cyclonedx`

2.10 Déploiement & Validation

Input	Type	Défaut	Description	Exemple
`deploy-enabled`	boolean	`false`	Activer déploiement	`true`
`deploy-env`	choice	-	Environnement : `dev` / `staging` / `prod`	`prod`
`deploy-command`	string	-	Commande déploiement personnalisée	`./scripts/deploy.sh`
`deploy-matrix`	string	-	Mapping env → version-suffix (JSON)	`{"dev":"-dev","staging":"-rc"}`
`pre-deploy-check`	boolean	`true`	Vérifier état repo (commits, tags)	`true`
`artifact-verification`	boolean	`true`	Vérifier intégrité artefacts (checksum)	`true`
`post-deploy-validation`	boolean	`false`	Valider déploiement (ping, logs)	`true`
`post-deploy-check-url`	string	-	URL à checker post-déploiement	`https://api.example.com/health`
`rollback-support`	boolean	`false`	Support auto-rollback en cas d'échec	`true`
`rollback-on-health-check-failure`	boolean	`false`	Rollback si health check échoue	`true`

2.11 Hooks & Plugins

Input	Type	Défaut	Description	Exemple
`webhook-pre-flight-url`	string	-	Webhook appelé avant toute action	`https://hooks.example.com/pre-flight`
`webhook-pre-flight-enabled`	boolean	`false`	Activer webhook pré-vol	`true`
`webhook-post-release-url`	string	-	Webhook après release complète	`https://hooks.example.com/release`
`webhook-post-release-enabled`	boolean	`false`	Activer webhook post-release	`true`
`webhook-on-failure-url`	string	-	Webhook en cas d'erreur	`https://hooks.example.com/error`
`webhook-on-failure-enabled`	boolean	`false`	Activer webhook d'erreur	`true`
`webhook-retry-attempts`	number	`3`	Tentatives webhook avant abandon	`5`
`webhook-timeout`	number	`30`	Timeout webhook (secondes)	`60`
`plugin-pre-flight-script`	string	-	Script shell exécuté avant actions	`./scripts/pre-flight.sh`
`plugin-pre-flight-enabled`	boolean	`false`	Activer script pré-vol	`true`
`plugin-post-release-script`	string	-	Script shell après release	`./scripts/post-release.sh`
`plugin-post-release-enabled`	boolean	`false`	Activer script post-release	`true`
`plugin-on-failure-script`	string	-	Script shell en cas d'erreur	`./scripts/on-error.sh`
`plugin-on-failure-enabled`	boolean	`false`	Activer script d'erreur	`true`
`plugin-timeout`	number	`300`	Timeout plugin (secondes)	`600`
`plugin-error-strategy`	choice	`warn`	Stratégie erreur plugin : `warn` / `error` / `ignore`	`error`

2.12 Rapports & Audit

Input	Type	Défaut	Description	Exemple
`generate-compliance-report`	boolean	`false`	Générer rapport audit complet	`true`

Input	Type	Défaut	Description	Exemple
`generate-deployment-report`	boolean	`false`	Générer rapport post-déploiement	`true`
`generate-pre-flight-summary`	boolean	`false`	Générer résumé Markdown pre-flight	`true`
`audit-log-export`	boolean	`false`	Exporter logs [AUDIT] en JSON	`true`
`audit-log-path`	string	`audit.json`	Chemin fichier audit	`logs/audit.json`

2.13 Gestion d'Erreurs & Retry

Input	Type	Défaut	Description	Exemple
`retry-on-failure`	boolean	`false`	Réessayer en cas d'échec	`true`
`retry-attempts`	number	`3`	Nombre max de tentatives	`5`
`retry-delay`	number	`5`	Délai entre tentatives (secondes)	`10`
`timeout-on-failure`	number	`0`	Timeout global (secondes, 0=illimité)	`1800`
`environment-lock`	boolean	`false`	Empêcher releases simultanées	`true`
`release-blockers-file`	string	`.release-blockers.yml`	Fichier conditions bloquantes	`config/blockers.yml`

3. OUTPUTS DE L'ACTION

3.1 Versioning & Tags

Output	Type	Description	Exemple
`current-version`	string	Version actuelle détectée/créée	`1.2.3`
`previous-version`	string	Version antérieure	`1.2.2`
`bump-type`	string	Type bump appliqué : `major` / `minor` / `patch` / `none`	`minor`
`tag-created`	boolean	Tag créé avec succès	`true`
`tag-name`	string	Nom complet du tag	`v1.2.3`
`tag-already-exists`	boolean	Tag existe déjà	`false`
`commits-since-last-tag`	number	Commits depuis dernier tag	`12`

Output	Type	Description	Exemple
`release-created`	boolean	GitHub Release créée	`true`
`release-url`	string	URL de la release GitHub	`https://github.com/.../releases/tag/v1.2.3`

3.2 Changelog & Release Notes

Output	Type	Description	Exemple
`changelog-generated`	boolean	Changelog généré/mis à jour	`true`
`changelog-content`	string	Contenu généré du changelog	`## [1.2.3] - 2024-01-15...`
`changelog-file-path`	string	Chemin fichier changelog	`CHANGELOG.md`
`release-notes`	string	Notes de release (dernière entrée)	`### Features\n- New API endpoint`

3.3 Build & Artifacts

Output	Type	Description	Exemple
`build-success`	boolean	Build réussi	`true`
`artifact-path`	string	Chemin artefacts générés	`dist/`
`checksum-file`	string	Chemin fichier checksum SHA256	`dist/release.sha256`
`version-file-path`	string	Chemin fichier VERSION créé	`VERSION`
`sbom-file-path`	string	Chemin fichier SBOM généré	`sbom.spdx.json`

3.4 Tests & Qualité

Output	Type	Description	Exemple
`tests-passed`	boolean	Tous les tests réussis	`true`
`coverage-percentage`	number	Couverture de code (%)	`92.5`
`coverage-meets-threshold`	boolean	Couverture ≥ seuil	`true`
`lint-passed`	boolean	Linting réussi	`true`
`lint-score`	number	Score lint (0-100)	`85`
`performance-regression-detected`	boolean	Régression perf détectée	`false`
`performance-comparison`	string	Comparaison perf vs version précédente	`2.3% faster`

3.5 Security & Scanning

Output	Type	Description	Exemple

Output	Type	Description	Exemple
`security-checks-passed`	boolean	Tous contrôles sécurité OK	`true`
`dependencies-vulnerable`	number	Dépendances vulnérables trouvées	`0`
`vulnerable-packages`	string	Liste packages vulnérables (JSON)	`[{"name":"lodash","severity":"high"}]`
`sast-issues`	number	Problèmes SAST détectés	`0`
`secrets-found`	number	Secrets détectés	`0`
`commit-signatures-valid`	boolean	Toutes signatures commits OK	`true`
`supply-chain-score`	number	Score supply chain (0-100)	`85`

3.6 Déploiement

Output	Type	Description	Exemple
`deployment-status`	string	Statut : `success` / `failure` / `pending`	`success`
`deployment-environment`	string	Environnement déployé	`prod`
`deployment-duration`	number	Durée déploiement (secondes)	`120`
`post-deploy-validation-passed`	boolean	Health check post-déploiement OK	`true`
`rollback-performed`	boolean	Rollback exécuté	`false`

3.7 Rapports & Audit

Output	Type	Description	Exemple
`compliance-report-path`	string	Chemin rapport audit	`reports/compliance.json`
`deployment-report-path`	string	Chemin rapport déploiement	`reports/deployment.json`
`audit-log-path`	string	Chemin export logs audit	`audit.json`
`pre-flight-summary-path`	string	Chemin résumé pre-flight	`artifacts/pre-flight.md`

3.8 Statuts Globaux

Output	Type	Description	Exemple
`workflow-status`	string	Statut final : `success` / `failure` / `partial`	`success`
`dryrun-mode`	boolean	Exécution en mode simulation	`true`
`execution-duration`	number	Durée exécution totale (secondes)	`420`
`commits-validated`	boolean	Commits respectent convention	`true`

---

4. SCOPES & MODULES (Groupés par Type)

4.1 Scope: GIT & CONFIG

Module: Git Configuration

- `git-user-name` (input: string)
- `git-user-email` (input: string)
- `auto-commit` (input: boolean)
- `auto-commit-message` (input: string)
- `auto-push` (input: boolean)
- `target-branch` (input: string)

Module: Config Management

- `config-source` (input: choice)
- `config-path` (input: string)
- `config-schema-check` (input: boolean)
- `load-config` (function)
- `config-lint` (function)

Module: Branching Strategy

- `trigger-branches` (input: string)
- `branch-protection-aware` (input: boolean)
- `branch-policy-check` (input: boolean)
- `review-enforcement` (input: boolean)

4.2 Scope: VERSIONING

Module: Version Detection & Bumping

- `detect-bump-type` (function)
- `bump` (input: choice)
- `version` (input: string - override)
- `initial` (input: string)
- `bump-type` (output)
- `current-version` (output)
- `previous-version` (output)
- `major-indicator` (input: pattern)
- `minor-indicator` (input: pattern)
- `patch-indicator` (input: pattern)

Module: Semantic Versioning

- `semantic-convention` (input: choice)
- `validate-semver` (function)
- `compare-versions` (function)
- `version-prefix` (input: string)
- `sync-version` (input: string - paths)

Module: Tag Management

- `create-tag` (function)
- `verify-tag` (function)

- `idempotent-check` (function)
- `tag-signature` (input: boolean)
- `tag-delete-on-failure` (input: boolean)
- `tag-name` (output)
- `tag-created` (output)

4.3 Scope: CHANGELOG & RELEASE

Module: Changelog Generation

- `generate-changelog` (function)
- `changelog-file` (input: path)
- `changelog-template` (input: path)
- `append-changelog` (input: choice)
- `extract-changelog` (function)
- `changelog-generated` (output)
- `changelog-content` (output)

Module: Release Notes

- `changelog-to-release-notes` (function)
- `release-notes` (output)
- `extract-changelog` (function)

Module: Release Creation

- `create-release` (function)
- `release-draft` (input: boolean)
- `release-prerelease` (input: boolean)
- `prerelease-suffix` (input: string)
- `release-created` (output)
- `release-url` (output)

Module: Multi-Package Release

- `package-release` (function)
- `monorepo-mode` (input: boolean)
- `detect-changed-packages` (function)

4.4 Scope: BUILD & ARTIFACTS

Module: Build Process

- `build-enabled` (input: boolean)
- `build-command` (input: string)
- `build-artifact-path` (input: path)
- `build-success` (output)
- `artifact-path` (output)

Module: Artifact Management

- `upload-artifact` (input: boolean)
- `artifact-retention-days` (input: number)

- `generate-version-file` (function)
- `generate-checksum` (function)
- `generate-sbom` (function)
- `artifact-signing` (function)
- `version-file-path` (output)
- `checksum-file` (output)
- `sbom-file-path` (output)

Module: Artifact Verification

- `artifact-verification` (input: boolean)
- `pre-deploy-check` (function)
- `pre-deploy-check-url` (input: string)

4.5 Scope: TESTING

Module: Unit & Integration Tests

- `tests-enabled` (input: boolean)
- `unit-tests-command` (input: string)
- `integration-tests-command` (input: string)
- `tests-passed` (output)

Module: Coverage Analysis

- `coverage-threshold` (input: number)
- `coverage-file-path` (input: path)
- `coverage-percentage` (output)
- `coverage-meets-threshold` (output)

Module: Code Quality

- `lint-enabled` (input: boolean)
- `lint-command` (input: string)
- `lint-report-parse` (function)
- `lint-report-path` (input: path)
- `lint-score-threshold` (input: number)
- `lint-passed` (output)
- `lint-score` (output)

Module: Performance Benchmarking

- `performance-benchmark-enabled` (input: boolean)
- `performance-benchmark-file` (input: path)
- `performance-regression-detected` (output)
- `performance-comparison` (output)

4.6 Scope: SECURITY

Module: Dependency Scanning

- `dependency-scan-enabled` (input: boolean)
- `dependency-scan-tool` (input: choice)

- `dependency-severity-threshold` (input: choice)
- `dependencies-vulnerable` (output)
- `vulnerable-packages` (output)

Module: Static Analysis (SAST)

- `sast-enabled` (input: boolean)
- `sast-tool` (input: choice)
- `sast-issues` (output)

Module: Secret Scanning

- `secret-scan-enabled` (input: boolean)
- `secrets-found` (output)

Module: Code Signing & Verification

- `commit-signature-check` (function)
- `commit-signature-check` (input: boolean)
- `tag-signature` (input: boolean)
- `gpg-key` (input: string - secret)
- `gpg-passphrase` (input: string - secret)
- `commit-signatures-valid` (output)

Module: Supply Chain Security

- `supply-chain-check` (function)
- `generate-sbom` (function)
- `sbom-format` (input: choice)
- `supply-chain-score` (output)

Module: Token & Authentication

- `github-token` (input: string - secret)
- `validate-token` (function)
- `token-scope-check` (function)
- `ssh-key` (input: string - secret)

4.7 Scope: DEPLOYMENT

Module: Deployment Execution

- `deploy-enabled` (input: boolean)
- `deploy-env` (input: choice)
- `deploy-command` (input: string)
- `deploy-matrix` (input: string - JSON)
- `deployment-status` (output)
- `deployment-environment` (output)
- `deployment-duration` (output)

Module: Post-Deployment Validation

- `post-deploy-validation` (input: boolean)
- `post-deploy-check-url` (input: string)

- `post-deploy-validation-passed` (output)

Module: Rollback Strategy

- `rollback-support` (input: boolean)
- `rollback-on-health-check-failure` (input: boolean)
- `rollback-performed` (output)

4.8 Scope: HOOKS & PLUGINS

Module: Webhooks

- `webhook-pre-flight-url` (input: string)
- `webhook-pre-flight-enabled` (input: boolean)
- `webhook-post-release-url` (input: string)
- `webhook-post-release-enabled` (input: boolean)
- `webhook-on-failure-url` (input: string)
- `webhook-on-failure-enabled` (input: boolean)
- `webhook-retry-attempts` (input: number)
- `webhook-timeout` (input: number)

Module: Plugin Scripts

- `plugin-pre-flight-script` (input: path)
- `plugin-pre-flight-enabled` (input: boolean)
- `plugin-post-release-script` (input: path)
- `plugin-post-release-enabled` (input: boolean)
- `plugin-on-failure-script` (input: path)
- `plugin-on-failure-enabled` (input: boolean)
- `plugin-timeout` (input: number)
- `plugin-error-strategy` (input: choice)

4.9 Scope: REPORTING & AUDIT

Module: Report Generation

- `generate-compliance-report` (function)
- `generate-deployment-report` (function)
- `generate-pre-flight-summary` (function)
- `compliance-report-path` (output)
- `deployment-report-path` (output)
- `pre-flight-summary-path` (output)

Module: Audit Logging

- `audit-log-export` (function)
- `audit-log-path` (input: path)
- `audit-log-path` (output)

4.10 Scope: ERROR HANDLING & VALIDATION

Module: Retry & Timeout

- `retry-on-failure` (input: boolean)
- `retry-attempts` (input: number)
- `retry-delay` (input: number)
- `timeout-on-failure` (input: number)

Module: Release Validation

- `ci-status-check` (function)
- `release-blockers-file` (input: path)
- `environment-lock` (input: boolean)

Module: Commit Validation

- `semantic-guard` (function)
- `commit-policy` (function)
- `commits-validated` (output)

---

5. FUNCTIONS & SCRIPTS (Ordre d'Exécution)

5.1 Phase: PRE-FLIGHT (Avant tout)

#	Fonction	Scope	Description	Inputs	Outputs	Condition
1	`load-config`	GIT & CONFIG	Charger fichier config	`config-source`, `config-path`	config_object	`config-source != default`
2	`config-lint`	GIT & CONFIG	Valider YAML config	config_object	`is_valid`	`config-schema-check == true`
3	`validate-token`	SECURITY	Vérifier token GitHub	`github-token`	`is_valid`, `scopes`	`validate-token == true`
4	`token-scope-check`	SECURITY	Contrôler permissions token	`github-token`	`has_permissions`	`token-scope-check == true`
5	`environment-lock`	ERROR HANDLING	Bloquer si release en cours	`.release.lock`	`is_locked`	`environment-lock == true`
6	`ci-status-check`	ERROR HANDLING	Vérifier CI jobs précédents	GitHub API	`all_success`	`ci-status-check == true`
7	`release-blockers`	ERROR HANDLING	Charger conditions bloquantes	`release-blockers-file`	`blockers_list`	-
8	**[WEBHOOK: pre-flight]**	HOOKS	Appeler webhook pré-vol	`webhook-pre-flight-url`	webhook_response	`webhook-pre-flight-enabled == true`

#	Fonction	Scope	Description	Inputs	Outputs	Condition
9	**[PLUGIN: pre-flight]**	HOOKS	Exécuter script pré-vol	`plugin-pre-flight-script`	script_output	`plugin-pre-flight-enabled == true`
10	`generate-pre-flight-summary`	REPORTING	Générer résumé pré-vol	all_config	`pre-flight-summary-path`	`generate-pre-flight-summary == true`

5.2 Phase: CONFIG & GIT SETUP

#	Fonction	Scope	Description	Inputs	Outputs	Condition
11	`git-config-user`	GIT & CONFIG	Configurer git user.name / user.email	`git-user-name`, `git-user-email`	git_configured	-
12	`checkout-code`	GIT & CONFIG	Checkout du code source	branch	code_ready	-
13	`branch-policy-check`	GIT & CONFIG	Vérifier stratégie merge	trigger branch	`policy_compliant`	`branch-policy-check == true`
14	`review-enforcement`	GIT & CONFIG	Bloquer si PR non approuvée	GitHub API	`is_approved`	`review-enforcement == true`

5.3 Phase: BUILD

#	Fonction	Scope	Description	Inputs	Outputs	Condition
15	`build-process`	BUILD	Exécuter build	`build-command`	`artifact-path`	`build-enabled == true`
16	`generate-checksum`	BUILD	Générer SHA256 des artefacts	`artifact-path`	`checksum-file`	`generate-checksum == true`
17	`generate-version-file`	BUILD	Créer VERSION/.version	`current-version`	`version-file-path`	`generate-version-file == true`
18	`artifact-verification`	BUILD	Vérifier intégrité artefacts	checksum	`artifact_verified`	`artifact-verification == true`

5.4 Phase: TESTS & QUALITY

#	Fonction	Scope	Description	Inputs	Outputs	Condition

#	Fonction	Scope	Description	Inputs	Outputs	Condition
19	`unit-tests`	TESTING	Exécuter tests unitaires	`unit-tests-command`	`tests-passed`	`tests-enabled == true`
20	`integration-tests`	TESTING	Exécuter tests d'intégration	`integration-tests-command`	tests_results	`integration-tests-command != empty`
21	`coverage-check`	TESTING	Analyser couverture de code	`coverage-file-path`, `coverage-threshold`	`coverage-percentage`, `coverage-meets-threshold`	`coverage-threshold > 0`
22	`lint-check`	TESTING	Exécuter linting	`lint-command`	`lint-passed`, `lint-score`	`lint-enabled == true`
23	`lint-report-parse`	TESTING	Parser rapport lint/sonar	`lint-report-path`	lint_issues	`lint-report-parse == true`
24	`performance-benchmark`	TESTING	Comparer perfs	`performance-benchmark-file`	`performance-regression-detected`, `performance-comparison`	`performance-benchmark-enabled == true`

5.5 Phase: SECURITY

#	Fonction	Scope	Description	Inputs	Outputs	Condition
25	`secret-scan`	SECURITY	Scanner secrets dans commits	trufflehog config	`secrets-found`	`secret-scan-enabled == true`
26	`dependency-scan`	SECURITY	Scan dépendances	`dependency-scan-tool`, `dependency-severity-threshold`	`dependencies-vulnerable`, `vulnerable-packages`	`dependency-scan-enabled == true`
27	`sast-analysis`	SECURITY	Analyse statique (SAST)	`sast-tool`	`sast-issues`	`sast-enabled == true`
28	`commit-signature-check`	SECURITY	Valider signatures GPG	git log	`commit-signatures-valid`	`commit-signature-check == true`
29	`supply-chain-check`	SECURITY	Vérifier supply chain	dépendances	`supply-chain-score`	`supply-chain-check == true`

#	Fonction	Scope	Description	Inputs	Outputs	Condition
30	`generate-sbom`	SECURITY	Générer SBOM	`sbom-format`	`sbom-file-path`	`generate-sbom == true`

5.6 Phase: VERSION & CHANGELOG

#	Fonction	Scope	Description	Inputs	Outputs	Condition
31	`get-commit-stats`	VERSIONING	Récupérer stats commits	last tag	`commits-stats`	-
32	`extract-breaking-changes`	VERSIONING	Détecter BREAKING CHANGE	commits	breaking_changes	-
33	`detect-bump-type`	VERSIONING	Scanner commits → bump auto	`major-indicator`, `minor-indicator`, `patch-indicator`	`bump-type`	`detect-bump-type == true`
34	`validate-semver`	VERSIONING	Valider format version	`version` ou `initial`	`is_valid`	-
35	`compare-versions`	VERSIONING	Comparer versions	current vs previous	comparison	-
36	`calculate-bump`	VERSIONING	Calculer nouvelle version	`bump-type` ou `version`	`current-version`	-
37	`sync-version`	VERSIONING	Sync version dans fichiers	`sync-version` paths, `current-version`	files_updated	`sync-version != empty`
38	`detect-changed-packages`	VERSIONING	Lister packages modifiés (monorepo)	git diff	changed_packages	`monorepo-mode == true`
39	`generate-changelog`	CHANGELOG	Générer changelog	commits, `changelog-template`	`changelog-generated`, `changelog-content`	`generate-changelog == true`
40	`changelog-to-release-notes`	CHANGELOG	Convertir changelog → release notes	`changelog-content`	`release-notes`	`changelog-to-release-notes == true`

#	Fonction	Scope	Description	Inputs	Outputs	Condition
41	`extract-changelog`	CHANGELOG	Extraire dernière entrée	`changelog-file`	`release-notes`	`extract-changelog == true`

5.7 Phase: TAGGING & RELEASE

#	Fonction	Scope	Description	Inputs	Outputs	Condition
42	`verify-tag`	VERSIONING	Vérifier si tag existe	`current-version`, `version-prefix`	`tag-already-exists`	`verify-tag == true`
43	`idempotent-check`	VERSIONING	Arrêter si tag existe	`tag-already-exists`	idempotent_check	`idempotent-check == true`
44	`create-tag`	VERSIONING	Créer tag Git	`current-version`, `tag-signature`	`tag-name`, `tag-created`	`create-tag == true`
45	`auto-commit`	GIT & CONFIG	Commit changelog/version	`auto-commit-message`	commit_sha	`auto-commit == true`
46	`auto-push`	GIT & CONFIG	Push commits et tag	`auto-push`, `branch-protection-aware`	push_success	`auto-push == true`
47	`create-release`	CHANGELOG	Créer GitHub Release	`tag-name`, `release-notes`, `release-draft`, `release-prerelease`	`release-created`, `release-url`	`create-release == true`
48	`package-release`	CHANGELOG	Release par package (monorepo)	changed_packages	releases_created	`package-release == true`
49	`upload-artifact`	BUILD	Upload changelog/version	`upload-artifact`	artifact_uploaded	`upload-artifact == true`

5.8 Phase: DEPLOYMENT (Optionnel)

#	Fonction	Scope	Description	Inputs	Outputs	Condition
50	`pre-deploy-check`	DEPLOYMENT	Vérifier repo avant déploiement	git status	`repo-clean`	`pre-deploy-check == true`

#	Fonction	Scope	Description	Inputs	Outputs	Condition
51	`deploy-process`	DEPLOYMENT	Exécuter déploiement	`deploy-command`, `deploy-env`, `deploy-matrix`	`deployment-status`, `deployment-duration`	`deploy-enabled == true`
52	`post-deploy-validation`	DEPLOYMENT	Valider déploiement (health check)	`post-deploy-check-url`	`post-deploy-validation-passed`	`post-deploy-validation == true`
53	`rollback-process`	DEPLOYMENT	Rollback auto en cas d'échec	previous tag	`rollback-performed`	`rollback-support == true` AND validation failed

5.9 Phase: REPORTING & CLEANUP

#	Fonction	Scope	Description	Inputs	Outputs	Condition
54	`generate-compliance-report`	REPORTING	Rapport audit complet	all_outputs	`compliance-report-path`	`generate-compliance-report == true`
55	`generate-deployment-report`	REPORTING	Rapport post-déploiement	deployment_data	`deployment-report-path`	`generate-deployment-report == true`
56	`audit-log-export`	REPORTING	Exporter logs [AUDIT] JSON	[AUDIT] logs	`audit-log-path`	`audit-log-export == true`
57	**[WEBHOOK: post-release]**	HOOKS	Appeler webhook post-release	`webhook-post-release-url`, all_outputs	webhook_response	`webhook-post-release-enabled == true`
58	**[PLUGIN: post-release]**	HOOKS	Exécuter script post-release	`plugin-post-release-script`, all_outputs	script_output	`plugin-post-release-enabled == true`

5.10 Phase: ERROR HANDLING (Si erreur détectée)

#	Fonction	Scope	Description	Inputs	Outputs	Condition
X1	`retry-logic`	ERROR HANDLING	Réessayer opération	`retry-attempts`, `retry-delay`	retry_result	`retry-on-failure == true`

#	Fonction	Scope	Description	Inputs	Outputs	Condition
X2	`tag-delete-on-failure`	VERSIONING	Supprimer tag créé	`tag-name`	tag_deleted	`tag-delete-on-failure == true` AND tag created
X3	**[WEBHOOK: on-failure]**	HOOKS	Appeler webhook d'erreur	`webhook-on-failure-url`, error_details	webhook_response	`webhook-on-failure-enabled == true`
X4	**[PLUGIN: on-failure]**	HOOKS	Exécuter script d'erreur	`plugin-on-failure-script`, error_details	script_output	`plugin-on-failure-enabled == true`

6. PLACEMENT DES HOOKS & PLUGINS DANS LE WORKFLOW

6.1 Timeline Complète avec Webhooks & Plugins

```
┌─ WORKFLOW START ──────────────────────────────────────────────────┐
│                                                                    │
│  [PRE-FLIGHT PHASE]                                                │
│  ├─ Load Config                                                    │
│  ├─ Validate Token                                                 │
│  ├─ Environment Lock                                               │
│  │                                                                 │
│  ├─► ⚡ WEBHOOK: pre-flight (si enabled)                           │
│  │   Payload: {config, token_valid, lock_status}                   │
│  │   Timeout: webhook-timeout (30s)                                │
│  │   Retry: webhook-retry-attempts (3)                             │
│  │                                                                 │
│  ├─► 🔌 PLUGIN: pre-flight (si enabled)                            │
│  │   Script: plugin-pre-flight-script                              │
│  │   Timeout: plugin-timeout (300s)                                │
│  │   On Error: plugin-error-strategy (warn/error/ignore)           │
│  │                                                                 │
│  ├─ Generate Pre-Flight Summary                                    │
│  └─ CI Status Check                                                │
│                                                                    │
│  [BUILD PHASE]                                                     │
│  ├─ Git User Config                                                │
│  ├─ Build Application                                              │
│  ├─ Generate Checksum                                              │
│  ├─ Artifact Verification                                          │
│  └─ Create Version File                                            │
│                                                                    │
│  [TEST PHASE]                                                      │
│  ├─ Unit Tests                                                     │
│  ├─ Integration Tests                                              │
│  ├─ Coverage Check                                                 │
│  ├─ Lint Check                                                     │
│  └─ Performance Benchmark                                          │
```

```
│                                                                  │
│  [SECURITY PHASE]                                                │
│  ├─ Secret Scan                                                  │
│  ├─ Dependency Scan                                              │
│  ├─ SAST Analysis                                                │
│  ├─ Commit Signature Check                                       │
│  ├─ Supply Chain Check                                           │
│  └─ Generate SBOM                                                │
│                                                                  │
│  [VERSION & CHANGELOG PHASE]                                     │
│  ├─ Get Commit Stats                                             │
│  ├─ Detect Bump Type                                             │
│  ├─ Calculate Version                                            │
│  ├─ Sync Version in Files                                        │
│  ├─ Generate Changelog                                           │
│  └─ Extract Release Notes                                        │
│                                                                  │
│  [TAGGING & RELEASE PHASE]                                       │
│  ├─ Verify Tag Doesn't Exist                                     │
│  ├─ Create Git Tag (+ GPG sign if enabled)                       │
│  ├─ Auto Commit (changelog/version files)                        │
│  ├─ Auto Push (commits + tag)                                    │
│  └─ Create GitHub Release                                        │
│                                                                  │
│  [DEPLOYMENT PHASE] (si deploy-enabled)                          │
│  ├─ Pre-Deploy Check (clean repo)                                │
│  ├─ Execute Deploy Command                                       │
│  ├─ Post-Deploy Validation (health check)                        │
│  └─ Rollback (si validation échoue)                              │
│                                                                  │
│  [POST-RELEASE PHASE]                                            │
│  ├─ Generate Reports (compliance, deployment)                    │
│  ├─ Export Audit Logs                                            │
│  ├─ Upload Artifacts                                             │
│  │                                                               │
│  ├─► ⚡ WEBHOOK: post-release (si enabled)                       │
│  │   Payload: {version, tag, release_url, all_stats}            │
│  │   Timeout: webhook-timeout (30s)                              │
│  │   Retry: webhook-retry-attempts (3)                           │
│  │                                                               │
│  ├─► 🔌 PLUGIN: post-release (si enabled)                        │
│  │   Script: plugin-post-release-script                          │
│  │   Args: VERSION TAG RELEASE_URL                               │
│  │   Timeout: plugin-timeout (300s)                              │
│  │   On Error: plugin-error-strategy (warn/error/ignore)         │
│  │                                                               │
│  └─ WORKFLOW COMPLETE ✓                                          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘

┌─ ERROR / FAILURE ──────────────────────────────────────────────┐
│                                                                  │
│  [ERROR DETECTED]                                                │
│  ├─ Retry Logic (si retry-on-failure)                            │
│  ├─ Rollback Changes (si applicable)                             │
│  ├─ Delete Tag (si tag-delete-on-failure)                        │
│  │                                                               │
│  ├─► ⚡ WEBHOOK: on-failure (si enabled)                         │
```

```
│  │   Payload: {error_message, phase, attempt, timestamp}        │
│  │   Timeout: webhook-timeout (30s)                              │
│  │   Retry: webhook-retry-attempts (3)                           │
│  │                                                                 │
│  ├─► 🔌 PLUGIN: on-failure (si enabled)                           │
│  │   Script: plugin-on-failure-script                            │
│  │   Args: ERROR_MESSAGE PHASE ATTEMPT                           │
│  │   Timeout: plugin-timeout (300s)                              │
│  │   On Error: ignore (pour ne pas masquer erreur principale)    │
│  │                                                                 │
│  └─ WORKFLOW FAILED ✗                                              │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

6.2 Détails des Payloads Webhooks

WEBHOOK: pre-flight

```json
{
  "event": "pre-flight",
  "timestamp": "2024-01-15T10:30:00Z",
  "repository": "owner/repo",
  "branch": "main",
  "config": {
    "version_prefix": "v",
    "initial_version": "0.1.0",
    "enable_deploy": false
  },
  "validation": {
    "token_valid": true,
    "lock_acquired": false,
    "ci_status": "success"
  },
  "environment": "production"
}
```

WEBHOOK: post-release

```json
{
  "event": "post-release",
  "timestamp": "2024-01-15T10:45:00Z",
  "repository": "owner/repo",
  "branch": "main",
  "release": {
    "version": "1.2.3",
    "tag": "v1.2.3",
    "url": "https://github.com/.../releases/tag/v1.2.3",
    "created_at": "2024-01-15T10:44:00Z"
  },
  "stats": {
    "commits_since_last": 12,
    "build_duration": 120,
    "tests_passed": true,
```

```
    "coverage": 92.5,
    "security_issues": 0,
    "deployment_status": "skipped"
  },
  "artifacts": [
    {"name": "build.tar.gz", "size": 2048576, "checksum": "sha256:..."}
  ]
}
```

WEBHOOK: on-failure

```
{
  "event": "on-failure",
  "timestamp": "2024-01-15T10:40:00Z",
  "repository": "owner/repo",
  "error": {
    "message": "Tests failed: 2 unit tests",
    "phase": "testing",
    "code": "TEST_FAILURE"
  },
  "context": {
    "attempt": 1,
    "max_attempts": 3,
    "retry_in_seconds": 5
  }
}
```

6.3 Paramètres des Plugins/Scripts

Plugin: pre-flight

```
#!/bin/bash
# Arguments passés en variables d'environnement
GITHUB_TOKEN="$1"
CONFIG_PATH="$2"
DEBUG_MODE="$3"

# Le script doit retourner:
# - Code 0: succès (continue)
# - Code 1: erreur bloquante (selon plugin-error-strategy)
# - Code 2: warning (toujours)

echo "[AUDIT] Pre-flight validation started"
exit 0
```

Plugin: post-release

```
#!/bin/bash
# Arguments positionnels
VERSION="$1"
TAG="$2"
```

```
RELEASE_URL="$3"
DRY_RUN="$4"

# Variables d'environnement disponibles
# - GITHUB_REPOSITORY
# - GITHUB_ACTOR
# - GITHUB_SHA
# - All action outputs

echo "[AUDIT] Publishing release notification..."
exit 0
```

---

7. MATRICE DE CONFIGURATION PAR CAS D'USAGE

7.1 Configuration Minimale (Quick Release)

```yaml
trigger-branches: main
auto-commit: true
auto-push: true
generate-changelog: true
create-tag: true
create-release: true
```

7.2 Configuration Monorepo (Multiple Packages)

```yaml
monorepo-mode: true
package-release: true
detect-changed-packages: true
sync-version: "package.json,packages/*/package.json,Chart.yaml"
deploy-matrix: '{"dev":"","staging":"-rc","prod":""}'
```

7.3 Configuration Sécurité Maximale

```yaml
secret-scan-enabled: true
dependency-scan-enabled: true
sast-enabled: true
commit-signature-check: true
tag-signature: true
supply-chain-check: true
generate-sbom: true
generate-compliance-report: true
```

7.4 Configuration Multi-Environnements avec Déploiement

```yaml
deploy-enabled: true
deploy-matrix: '{"dev":"-dev","staging":"-rc","prod":""}'
post-deploy-validation: true
```

```
  post-deploy-check-url: "https://api.example.com/health"
  rollback-support: true
```

---

8. EXEMPLE D'UTILISATION COMPLÈTE

action.yml

```yaml
name: 'Instant Release - CI/CD Framework'
description: 'Modular GitHub Action for complete CI/CD automation'

inputs:
  # Git & Config (REQUIRED)
  github-token:
    description: 'GitHub token with contents:write permission'
    required: true
  git-user-name:
    description: 'Git user name for commits'
    default: 'github-actions[bot]'
  git-user-email:
    description: 'Git user email'
    default: 'github-actions[bot]@github.com'

  # Versioning
  bump:
    description: 'Version bump type: auto|major|minor|patch'
    default: 'auto'
  generate-changelog:
    description: 'Generate changelog'
    default: 'true'
  create-tag:
    description: 'Create Git tag'
    default: 'true'
  create-release:
    description: 'Create GitHub Release'
    default: 'true'

  # Build
  build-enabled:
    description: 'Enable build step'
    default: 'true'
  build-command:
    description: 'Build command'
    default: 'npm run build'

  # Tests
  tests-enabled:
    description: 'Enable tests'
    default: 'true'
  unit-tests-command:
    description: 'Unit tests command'
    default: 'npm test'
  coverage-threshold:
    description: 'Minimum coverage percentage'
    default: '80'
```

```yaml
    # Security
    dependency-scan-enabled:
      description: 'Enable dependency scanning'
      default: 'true'
    secret-scan-enabled:
      description: 'Enable secret scanning'
      default: 'false'

    # Deployment
    deploy-enabled:
      description: 'Enable deployment'
      default: 'false'
    deploy-command:
      description: 'Deploy command'

    # Webhooks & Plugins
    webhook-post-release-enabled:
      description: 'Enable post-release webhook'
      default: 'false'
    webhook-post-release-url:
      description: 'Post-release webhook URL'
    plugin-post-release-enabled:
      description: 'Enable post-release plugin'
      default: 'false'
    plugin-post-release-script:
      description: 'Post-release plugin script path'

    # Debug & Dry-run
    debug:
      description: 'Enable debug mode'
      default: 'false'
    dry-run:
      description: 'Enable dry-run mode'
      default: 'false'

  outputs:
    current-version:
      description: 'Current version created'
      value: ${{ steps.version.outputs.current-version }}
    bump-type:
      description: 'Bump type applied'
      value: ${{ steps.version.outputs.bump-type }}
    tag-created:
      description: 'Whether tag was created'
      value: ${{ steps.tag.outputs.tag-created }}
    changelog-generated:
      description: 'Whether changelog was generated'
      value: ${{ steps.changelog.outputs.changelog-generated }}
    release-url:
      description: 'GitHub Release URL'
      value: ${{ steps.release.outputs.release-url }}
    deployment-status:
      description: 'Deployment status'
      value: ${{ steps.deploy.outputs.deployment-status }}
```

```yaml
  runs:
    using: 'composite'
    steps:
      # Pre-flight
      - name: Pre-flight checks
        run: ${{ github.action_path }}/scripts/pre-flight.sh
        shell: bash
        env:
          GITHUB_TOKEN: ${{ inputs.github-token }}
          DEBUG: ${{ inputs.debug }}
          DRY_RUN: ${{ inputs.dry-run }}

      # Webhook pre-flight
      - name: Webhook pre-flight
        if: ${{ inputs.webhook-pre-flight-enabled == 'true' }}
        run: ${{ github.action_path }}/scripts/webhook-call.sh
        shell: bash
        env:
          WEBHOOK_URL: ${{ inputs.webhook-pre-flight-url }}
          WEBHOOK_EVENT: 'pre-flight'
          WEBHOOK_TIMEOUT: ${{ inputs.webhook-timeout }}

      # Plugin pre-flight
      - name: Plugin pre-flight
        if: ${{ inputs.plugin-pre-flight-enabled == 'true' }}
        run: |
          bash "${{ inputs.plugin-pre-flight-script }}" \
            "${{ inputs.github-token }}" \
            "${{ inputs.config-path }}" \
            "${{ inputs.debug }}"
        shell: bash
        timeout-minutes: ${{ inputs.plugin-timeout }}

      # Build
      - name: Build
        if: ${{ inputs.build-enabled == 'true' }}
        run: ${{ inputs.build-command }}
        shell: bash

      # Tests
      - name: Unit Tests
        if: ${{ inputs.tests-enabled == 'true' }}
        run: ${{ inputs.unit-tests-command }}
        shell: bash

      - name: Coverage Check
        if: ${{ inputs.tests-enabled == 'true' }}
        run: ${{ github.action_path }}/scripts/coverage-check.sh
        shell: bash
        env:
          COVERAGE_THRESHOLD: ${{ inputs.coverage-threshold }}
          COVERAGE_FILE: ${{ inputs.coverage-file-path }}

      - name: Lint
        if: ${{ inputs.lint-enabled == 'true' }}
        run: ${{ inputs.lint-command }}
        shell: bash
```

```yaml
    # Security
    - name: Dependency Scan
      if: ${{ inputs.dependency-scan-enabled == 'true' }}
      run: ${{ github.action_path }}/scripts/dependency-scan.sh
      shell: bash
      env:
        SCAN_TOOL: ${{ inputs.dependency-scan-tool }}
        SEVERITY_THRESHOLD: ${{ inputs.dependency-severity-threshold }}

    - name: Secret Scan
      if: ${{ inputs.secret-scan-enabled == 'true' }}
      run: ${{ github.action_path }}/scripts/secret-scan.sh
      shell: bash

    - name: SAST Analysis
      if: ${{ inputs.sast-enabled == 'true' }}
      run: ${{ github.action_path }}/scripts/sast-analysis.sh
      shell: bash
      env:
        SAST_TOOL: ${{ inputs.sast-tool }}

    # Versioning
    - name: Calculate Version
      id: version
      run: ${{ github.action_path }}/scripts/calculate-version.sh
      shell: bash
      env:
        BUMP: ${{ inputs.bump }}
        VERSION: ${{ inputs.version }}
        INITIAL: ${{ inputs.initial }}
        MAJOR_INDICATOR: ${{ inputs.major-indicator }}
        MINOR_INDICATOR: ${{ inputs.minor-indicator }}
        PATCH_INDICATOR: ${{ inputs.patch-indicator }}

    - name: Sync Version Files
      if: ${{ inputs.sync-version != '' }}
      run: ${{ github.action_path }}/scripts/sync-version.sh
      shell: bash
      env:
        VERSION: ${{ steps.version.outputs.current-version }}
        SYNC_PATHS: ${{ inputs.sync-version }}

    # Changelog
    - name: Generate Changelog
      if: ${{ inputs.generate-changelog == 'true' }}
      id: changelog
      run: ${{ github.action_path }}/scripts/generate-changelog.sh
      shell: bash
      env:
        CHANGELOG_FILE: ${{ inputs.changelog-file }}
        CHANGELOG_TEMPLATE: ${{ inputs.changelog-template }}
        VERSION: ${{ steps.version.outputs.current-version }}

    # Tagging & Release
    - name: Create Tag
      if: ${{ inputs.create-tag == 'true' }}
      id: tag
      run: ${{ github.action_path }}/scripts/create-tag.sh
```

```
        shell: bash
        env:
          VERSION: ${{ steps.version.outputs.current-version }}
          VERSION_PREFIX: ${{ inputs.version-prefix }}
          TAG_SIGNATURE: ${{ inputs.tag-signature }}
          GPG_KEY: ${{ inputs.gpg-key }}
          DRY_RUN: ${{ inputs.dry-run }}

      - name: Auto Commit
        if: ${{ inputs.auto-commit == 'true' && inputs.create-tag == 'true' }}
        run: ${{ github.action_path }}/scripts/auto-commit.sh
        shell: bash
        env:
          GIT_USER_NAME: ${{ inputs.git-user-name }}
          GIT_USER_EMAIL: ${{ inputs.git-user-email }}
          COMMIT_MESSAGE: ${{ inputs.auto-commit-message }}
          VERSION: ${{ steps.version.outputs.current-version }}
          DRY_RUN: ${{ inputs.dry-run }}

      - name: Auto Push
        if: ${{ inputs.auto-push == 'true' && inputs.create-tag == 'true' }}
        run: ${{ github.action_path }}/scripts/auto-push.sh
        shell: bash
        env:
          GITHUB_TOKEN: ${{ inputs.github-token }}
          TARGET_BRANCH: ${{ inputs.target-branch }}
          DRY_RUN: ${{ inputs.dry-run }}

      - name: Create Release
        if: ${{ inputs.create-release == 'true' && inputs.create-tag == 'true' }}
        id: release
        run: ${{ github.action_path }}/scripts/create-release.sh
        shell: bash
        env:
          GITHUB_TOKEN: ${{ inputs.github-token }}
          TAG: ${{ steps.tag.outputs.tag-name }}
          RELEASE_NOTES: ${{ steps.changelog.outputs.release-notes }}
          DRAFT: ${{ inputs.release-draft }}
          PRERELEASE: ${{ inputs.release-prerelease }}
          DRY_RUN: ${{ inputs.dry-run }}

      # Deployment
      - name: Pre-Deploy Check
        if: ${{ inputs.deploy-enabled == 'true' }}
        run: ${{ github.action_path }}/scripts/pre-deploy-check.sh
        shell: bash

      - name: Deploy
        if: ${{ inputs.deploy-enabled == 'true' }}
        id: deploy
        run: ${{ inputs.deploy-command }}
        shell: bash
        env:
          ENVIRONMENT: ${{ inputs.deploy-env }}
          VERSION: ${{ steps.version.outputs.current-version }}
          DRY_RUN: ${{ inputs.dry-run }}

      - name: Post-Deploy Validation
```

```
      if: ${{ inputs.post-deploy-validation == 'true' && inputs.deploy-enabled == 'true'
}}
      run: ${{ github.action_path }}/scripts/post-deploy-validation.sh
      shell: bash
      env:
        CHECK_URL: ${{ inputs.post-deploy-check-url }}
        TIMEOUT: 300

    # Reporting
    - name: Generate Reports
      if: ${{ always() }}
      run: ${{ github.action_path }}/scripts/generate-reports.sh
      shell: bash
      env:
        GENERATE_COMPLIANCE: ${{ inputs.generate-compliance-report }}
        GENERATE_DEPLOYMENT: ${{ inputs.generate-deployment-report }}
        VERSION: ${{ steps.version.outputs.current-version }}

    # Webhook post-release
    - name: Webhook post-release
      if: ${{ success() && inputs.webhook-post-release-enabled == 'true' }}
      run: ${{ github.action_path }}/scripts/webhook-call.sh
      shell: bash
      env:
        WEBHOOK_URL: ${{ inputs.webhook-post-release-url }}
        WEBHOOK_EVENT: 'post-release'
        VERSION: ${{ steps.version.outputs.current-version }}
        TAG: ${{ steps.tag.outputs.tag-name }}
        RELEASE_URL: ${{ steps.release.outputs.release-url }}

    # Plugin post-release
    - name: Plugin post-release
      if: ${{ success() && inputs.plugin-post-release-enabled == 'true' }}
      run: |
        bash "${{ inputs.plugin-post-release-script }}" \
          "${{ steps.version.outputs.current-version }}" \
          "${{ steps.tag.outputs.tag-name }}" \
          "${{ steps.release.outputs.release-url }}" \
          "${{ inputs.dry-run }}"
      shell: bash
      timeout-minutes: ${{ inputs.plugin-timeout }}

    # Error handling - Webhook on-failure
    - name: Webhook on-failure
      if: ${{ failure() && inputs.webhook-on-failure-enabled == 'true' }}
      run: ${{ github.action_path }}/scripts/webhook-call.sh
      shell: bash
      env:
        WEBHOOK_URL: ${{ inputs.webhook-on-failure-url }}
        WEBHOOK_EVENT: 'on-failure'
        ERROR_MESSAGE: ${{ job.status }}
        PHASE: 'workflow'

    # Error handling - Plugin on-failure
    - name: Plugin on-failure
      if: ${{ failure() && inputs.plugin-on-failure-enabled == 'true' }}
      run: |
        bash "${{ inputs.plugin-on-failure-script }}" \
```

```
            "${{ job.status }}" \
            "workflow" \
            "1"
        shell: bash
        timeout-minutes: ${{ inputs.plugin-timeout }}
        continue-on-error: true

      # Cleanup
      - name: Upload Artifacts
        if: ${{ always() && inputs.upload-artifact == 'true' }}
        uses: actions/upload-artifact@v3
        with:
          name: release-artifacts
          path: |
            ${{ inputs.changelog-file }}
            ${{ steps.version.outputs.version-file-path }}
          retention-days: ${{ inputs.artifact-retention-days }}
```

---

9. GUIDE D'INTÉGRATION DANS UN WORKFLOW

9.1 Workflow Simple (Automatique)

```
name: Release

on:
  push:
    branches: [ main ]

jobs:
  release:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      packages: write

    steps:
      - uses: actions/checkout@v3
        with:
          fetch-depth: 0

      - name: Release
        uses: ./
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          build-command: npm run build
          unit-tests-command: npm test
```

9.2 Workflow Avancé (Avec Webhooks & Plugins)

```
name: Advanced Release

on:
```

```yaml
  push:
    branches: [ main ]
  workflow_dispatch:
    inputs:
      bump:
        description: 'Version bump'
        required: false
        default: 'auto'

jobs:
  release:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      packages: write

    steps:
      - uses: actions/checkout@v3
        with:
          fetch-depth: 0

      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: 18

      - name: Release with Webhooks
        uses: ./
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          bump: ${{ github.event.inputs.bump || 'auto' }}

          # Build & Tests
          build-command: npm run build
          unit-tests-command: npm test
          coverage-threshold: 85

          # Security
          dependency-scan-enabled: 'true'
          secret-scan-enabled: 'true'
          sast-enabled: 'true'
          sast-tool: codeql

          # Deployment
          deploy-enabled: 'true'
          deploy-command: npm run deploy
          deploy-env: production
          post-deploy-validation: 'true'
          post-deploy-check-url: https://api.example.com/health

          # Webhooks
          webhook-post-release-enabled: 'true'
          webhook-post-release-url: ${{ secrets.WEBHOOK_URL }}

          # Plugins
          plugin-post-release-enabled: 'true'
          plugin-post-release-script: ./scripts/post-release.sh
```

```
            # Reporting
            generate-compliance-report: 'true'
            generate-deployment-report: 'true'
            audit-log-export: 'true'
```

---

10. STRUCTURE DES FICHIERS DU PROJET

```
instantrelease/
├── action.yml                          # Définition action
├── README.md                           # Documentation principale
├── SPECIFICATION.md                    # Cette documentation
│
├── scripts/
│   ├── lib/                           # Librairies réutilisables
│   │   ├── logger.sh                 # Logging avec [AUDIT], [DEBUG]
│   │   ├── utils.sh                  # Fonctions utilitaires
│   │   ├── git.sh                    # Opérations Git
│   │   ├── github-api.sh             # Interactions GitHub API
│   │   └── validators.sh             # Validations (semver, yaml, etc.)
│   │
│   ├── pre-flight.sh                 # Phase pré-vol
│   ├── webhook-call.sh               # Appels webhooks
│   │
│   ├── build/
│   │   ├── build.sh                  # Build application
│   │   ├── generate-checksum.sh      # Génération checksums
│   │   └── generate-version-file.sh  # Création VERSION file
│   │
│   ├── tests/
│   │   ├── unit-tests.sh             # Tests unitaires
│   │   ├── coverage-check.sh         # Vérification couverture
│   │   ├── lint.sh                   # Linting
│   │   └── performance-benchmark.sh  # Benchmarks
│   │
│   ├── security/
│   │   ├── dependency-scan.sh        # Scan dépendances
│   │   ├── secret-scan.sh            # Scan secrets
│   │   ├── sast-analysis.sh          # Analyse SAST
│   │   ├── commit-signature-check.sh # Vérification signatures
│   │   └── supply-chain-check.sh     # Vérification supply chain
│   │
│   ├── versioning/
│   │   ├── calculate-version.sh      # Calcul version
│   │   ├── detect-bump-type.sh       # Détection bump
│   │   ├── validate-semver.sh        # Validation semver
│   │   └── sync-version.sh           # Sync fichiers version
│   │
│   ├── changelog/
│   │   ├── generate-changelog.sh     # Génération changelog
│   │   ├── extract-changelog.sh      # Extraction dernière entrée
│   │   └── changelog-to-release-notes.sh # Conversion
│   │
│   ├── release/
│   │   ├── create-tag.sh             # Création tag
```

```
│   │   ├── verify-tag.sh             # Vérification tag
│   │   ├── auto-commit.sh            # Commit auto
│   │   ├── auto-push.sh              # Push auto
│   │   └── create-release.sh         # Création GitHub Release
│   │
│   ├── deploy/
│   │   ├── pre-deploy-check.sh       # Pré-déploiement
│   │   ├── post-deploy-validation.sh # Post-déploiement
│   │   └── rollback.sh               # Rollback
│   │
│   └── reporting/
│       ├── generate-reports.sh       # Génération rapports
│       ├── compliance-report.sh      # Rapport audit
│       └── audit-log-export.sh       # Export logs audit
│
├── templates/
│   ├── changelog-default.hbs         # Template changelog défaut
│   ├── compliance-report.hbs         # Template rapport
│   └── pre-flight-summary.hbs        # Template résumé pré-vol
│
└── tests/
    ├── unit/                         # Tests unitaires des scripts
    └── integration/                  # Tests d'intégration
```

---

11. RÉSUMÉ EXÉCUTIF

Points Clés de l'Architecture

- ☑ **Modularité** : Chaque scope est indépendant et can-disable
- ☑ **Robustesse** : Retry logic, timeout, error handling complets
- ☑ **Transparence** : Logs [AUDIT], [DEBUG], rapports détaillés
- ☑ **Extensibilité** : Webhooks + Plugins à 3 moments clés
- ☑ **Sécurité** : Scanning complet, signing, supply chain
- ☑ **Monorepo-Ready** : Support multi-packages avec versioning indépendant
- ☑ **Cloud-Native** : Conçu pour GitHub Actions, intégrable ailleurs

Timeline d'Exécution Typique

```
PRE-FLIGHT (30s)
  ↓
BUILD (120s)
  ↓
TESTS + QUALITY (180s)
  ↓
SECURITY (120s)
  ↓
VERSIONING + CHANGELOG (30s)
  ↓
TAGGING + RELEASE (15s)
  ↓
DEPLOYMENT (300s, si enabled)
  ↓
REPORTING + WEBHOOKS (30s)
```

```
    ↓
  TOTAL: 5-10 minutes
```